SEMINARIO FLEX

Análisis léxico con FLEX

Modelos de Computación

Expresiones regulares y un analizador en C++

El analizador léxico

Lee caracteres y reconoce **tokens** que puede consumir un analizador sintáctico.

```
Entrada:    total = 12 + 3;
Tokens:     ID  ASIGNACIÓN  ENTERO  SUMA  ENTERO  PUNTO_Y_COMA
```

Lexema: texto reconocido, como `total` o `12`.

Token: categoría de ese texto, como `ID` o `ENTERO`.

LEX y FLEX

LEX es un generador de analizadores léxicos: recibe expresiones regulares con acciones y produce código C que reconoce esos patrones.

FLEX es una alternativa libre a LEX, compatible con gran parte de su especificación. Puede generar analizadores en C o clases de C++.

1. Escribimos una especificación en un fichero `.l`.
2. FLEX genera el código del analizador.
3. Un compilador C++ produce el ejecutable.

> Usaremos la interfaz de clases de C++: `flex -+`, `lex.yy.cc` y `yyFlexLexer`.

Patrones básicos

Patrón	Qué reconoce
`[A-Z]`	Una letra mayúscula ASCII
`[a-zA-Z0-9]`	Una letra ASCII o un dígito
`[0-9]{5}`	Cinco dígitos consecutivos
`abc.*`	`abc` seguido de caracteres hasta antes de `\n`
`"int"|"INT"`	Una de las dos cadenas literales
`.` / `\.`	Cualquier carácter salvo `\n` / un punto literal

> FLEX reconoce desde la posición actual. Un patrón no valida automáticamente toda la entrada.

Repetición y agrupación

Patrón	Significado	Ejemplo
`r*`	Cero o más repeticiones	`[0-9]*`
`r+`	Una o más repeticiones	`[0-9]+`
`r?`	Cero o una aparición	`-?[0-9]+`
`(r|s)`	Alternativa agrupada	`(ab|cd)+`
`rs`	Concatenación	`[a-z][0-9]`

> En FLEX, `+` indica repetición. La unión se escribe `|`, aunque en la teoría hayamos usado `+`.

Cómo elige FLEX una regla

1. Elige la coincidencia que consume **más caracteres**.
2. Si varias consumen lo mismo, elige la **primera regla** del fichero.

```
"if"                    { std::cout << "IF\n"; }
[a-zA-Z_][a-zA-Z0-9_]*   { std::cout << "ID\n"; }
```

Cómo elige FLEX una regla

1. Elige la coincidencia que consume **más caracteres**.
2. Si varias consumen lo mismo, elige la **primera regla** del fichero.

```
"if"                    { std::cout << "IF\n"; }
[a-zA-Z_][a-zA-Z0-9_]*   { std::cout << "ID\n"; }
```

`if` : empate de longitud, gana la primera regla.
`ifx` : gana el identificador, con tres caracteres.

Predicción de tokens

Supón estas reglas, en este orden:

```
[0-9]+          { std::cout << "ENTERO\n"; }
[0-9]+"."[0-9]+ { std::cout << "REAL\n"; }
```

¿Qué reconoce al leer `12.5` ?

Predicción de tokens

Supón estas reglas, en este orden:

```
[0-9]+          { std::cout << "ENTERO\n"; }
[0-9]+"."[0-9]+ { std::cout << "REAL\n"; }
```

¿Qué reconoce al leer `12.5` ?

REAL: consume cuatro caracteres. El entero solo consumiría dos.

Predicción de tokens

Supón estas reglas, en este orden:

```
[0-9]+          { std::cout << "ENTERO\n"; }
[0-9]+"."[0-9]+ { std::cout << "REAL\n"; }
```

¿Qué reconoce al leer `12.5`?

REAL: consume cuatro caracteres. El entero solo consumiría dos.

> El orden decide los empates. La longitud tiene prioridad.

Las tres secciones del fichero

```
/* Declaraciones, opciones y definiciones */
%%
/* Patrones y acciones */
%%
/* Código de usuario: main y funciones auxiliares */
```

Los separadores `%%` van solos en su línea.

> Modificamos el fichero `.l` y volvemos a generar el analizador.

Declaraciones y definiciones

```
%option c++ noyywrap
%{
#include <iostream>
#include <fstream>
%}
DIGITO  [0-9]
ID      [a-zA-Z_][a-zA-Z0-9_]*
DECIMAL {DIGITO}+"."{DIGITO}+
```

El bloque `%{ ... %}` incorpora código C++ al fichero generado.

Las definiciones se reutilizan con llaves: `{DIGITO}`.

Reglas y acciones

```
{ID}       { std::cout << "ID: " << YYText() << '\n'; }
[ \t\n]+ { /* Ignorar separadores */ }
.          { std::cerr << "No reconocido: "
                       << YYText() << '\n'; }
```

El patrón comienza al principio de la línea. **Espacios o tabuladores** lo separan de la acción.

> Sin regla explícita, FLEX copia el carácter no reconocido a la salida. Esto también puede ocurrir con un salto de línea.

La información del lexema

Interfaz C++	Utilidad
`YYText()`	Texto que acaba de reconocer
`YYLeng()`	Longitud de ese texto en bytes
`lineno()`	Línea actual, con `%option yylineno`
`yylex()`	Ejecuta el reconocimiento

> En nuestras acciones, que no devuelven tokens con `return`, una llamada a `yylex()` procesa hasta el final de la entrada.

Final de entrada y código de usuario

`%option noyywrap` evita tener que proporcionar `yywrap()`.

Después del segundo `%%` escribimos `main` y las funciones auxiliares.

```
int main() {
    yyFlexLexer analizador;
    analizador.yylex();
    return 0;
}
```

Por defecto, el objeto lee de la entrada estándar.

Ejemplo: contador de texto

Queremos contar **palabras**, **bytes** y **saltos de línea**.

```
hola mundo\n
flex\n
```

Una palabra será una secuencia sin espacios, tabuladores ni `\n`.

> Usamos texto ASCII y finales de línea LF. Contar `\n` no equivale siempre a contar líneas de texto.

Ejemplo: contador de texto

Queremos contar **palabras**, **bytes** y **saltos de línea**.

```
hola mundo\n
flex\n
```

Una palabra será una secuencia sin espacios, tabuladores ni `\n`.

---

3 palabras · 16 bytes · 2 saltos de línea

> Usamos texto ASCII y finales de línea LF. Contar `\n` no equivale siempre a contar líneas de texto.

Contador: declaraciones

```
%option c++ noyywrap
%{
#include <iostream>
#include <fstream>
#include <cstddef>
std::size_t bytes = 0, palabras = 0, saltos = 0;
void escribir_datos();
%}
```

Los contadores empiezan a cero. Declaramos la función antes de utilizarla.

Contador: reglas

```
%%
[^ \t\n]+ { ++palabras; bytes += YYLeng(); }
[ \t]+    { bytes += YYLeng(); }
\n        { ++saltos; ++bytes; }
%%
```

Contador: reglas

```
%%
[^ \t\n]+ { ++palabras; bytes += YYLeng(); }
[ \t]+    { bytes += YYLeng(); }
\n        { ++saltos; ++bytes; }
%%
```

Primera regla: cuenta una palabra y suma su longitud.

Contador: reglas

```
%%
[^ \t\n]+ { ++palabras; bytes += YYLeng(); }
[ \t]+    { bytes += YYLeng(); }
\n        { ++saltos; ++bytes; }
%%
```

Primera regla: cuenta una palabra y suma su longitud.

Segunda regla: suma espacios y tabuladores.

Contador: reglas

```
%%
[^ \t\n]+ { ++palabras; bytes += YYLeng(); }
[ \t]+    { bytes += YYLeng(); }
\n        { ++saltos; ++bytes; }
%%
```

Primera regla: cuenta una palabra y suma su longitud.

Segunda regla: suma espacios y tabuladores.

Tercera regla: cuenta cada salto de línea y su byte.

Contador: apertura y análisis

```
int main(int argc, char* argv[]) {
    if (argc != 2) {
        std::cerr << "Uso: ./contador entrada.txt\n";
        return 1;
    }
    std::ifstream fichero(argv[1], std::ios::binary);
    if (!fichero) {
        std::cerr << "No se puede abrir el fichero\n";
        return 1;
    }
    yyFlexLexer analizador(&fichero);
    analizador.yylex();
    escribir_datos();
    return 0;
}
```

Contador: resultado y compilación

```
void escribir_datos() {
    std::cout << "Palabras: " << palabras << '\n'
              << "Bytes: " << bytes << '\n'
              << "Saltos: " << saltos << '\n';
}
```

```
flex -+ -o lex.yy.cc contador.l
g++ -std=c++11 lex.yy.cc -o contador
./contador entrada.txt
```

> Ejecutamos estos comandos en la carpeta `materiales`. `flex++` también activa el modo C++.

Pruebas que explican el resultado

Entrada	Palabras	Bytes	Saltos
Fichero vacío	0	0	0
`hola` sin LF final	1	4	0
`hola\n`	1	5	1
`a\tb\n`	2	4	1
`hola mundo\nflex\n`	3	16	2

> Con UTF-8, una letra acentuada puede ocupar varios bytes. Este programa cuenta bytes, no caracteres Unicode.

Ejercicio: reconocer tokens

En parejas, completad `tokens-inicio.l` para reconocer:

- La palabra reservada `if`.
- Reales con dígitos a ambos lados del punto.
- Un mensaje para cada carácter no reconocido.

```
if ifx 12 12.5 @
```

> Disponéis de 8 minutos. Antes de ejecutar, escribid la salida esperada y decidid dónde colocar cada regla.

Solución: patrones y orden

```
"if"           { std::cout << "IF\n"; }
{ID}           { std::cout << "ID " << YYText() << '\n'; }
{DIGITO}+      { std::cout << "ENTERO " << YYText() << '\n'; }
{REAL}         { std::cout << "REAL " << YYText() << '\n'; }
[ \t\n]+     { /* Ignorar separadores */ }
.              { std::cout << "ERROR " << YYText() << '\n'; }
```

`REAL` se define como `{DIGITO}+"."{DIGITO}+`.

> La palabra reservada precede al identificador. La regla de error queda al final.

Solución: salida y explicación

```
IF
ID ifx
ENTERO 12
REAL 12.5
ERROR @
```

`if` : empate, gana la palabra reservada.

`ifx` y `12.5` : gana la coincidencia de mayor longitud.

`@` : lo captura la regla de error.

Errores habituales

- Escribir `yylength` en lugar de `yyleng` en la interfaz C.
- Esperar que `.` reconozca también el salto de línea.
- Colocar una palabra reservada después del identificador.
- Confundir reconocimiento de fragmentos con validación completa.
- Mezclar `FILE*` de C con los flujos de `yyFlexLexer`.

> Comprobación rápida: con las reglas del ejercicio, `12abc` produce un entero y un identificador.

Comprobación final

1. ¿Qué dos criterios usa FLEX para elegir una regla?
2. ¿Qué diferencia hay entre `YYText()` y `YYLeng()`?
3. ¿Qué debes ejecutar después de modificar el `.l`?

El apéndice siguiente queda como material de consulta.

Comprobación final

1. ¿Qué dos criterios usa FLEX para elegir una regla?
2. ¿Qué diferencia hay entre `YYText()` y `YYLeng()`?
3. ¿Qué debes ejecutar después de modificar el `.l`?

Mayor longitud y, en empate, primera regla.
Texto reconocido y longitud en bytes.
FLEX, compilación y una prueba de ejecución.

El apéndice siguiente queda como material de consulta.

Consulta: clases y repeticiones

Sintaxis	Significado
`[abc]` / `[^abc]`	Un carácter incluido / no incluido en la clase
`[a-z]`	Rango dentro de una clase
`r{n}` / `r{n,}`	Exactamente n / al menos n repeticiones
`r{0,m}` / `r{n,m}`	Hasta m / entre n y m repeticiones
`{nombre}`	Expande una definición de patrón
`"texto"`	Reconoce esos caracteres literalmente

> Una clase negada puede incluir `\n`. Para excluirlo, escríbelo dentro de la clase: `[^\n]`.

Consulta: anclas y escapes

Sintaxis	Significado
`^r`	r al comienzo de línea
`r$`	r justo antes de un salto de línea
`\n` / `\t`	Salto de línea / tabulación
`\.` / `\+`	Punto / signo más literal
`(r)`	Agrupación
`.*`	Cero o más caracteres distintos de LF

> El significado de un metacarácter depende del contexto. Por ejemplo, `^` niega una clase si aparece al inicio de sus corchetes.

Consulta: interfaces C y C++

Operación	Interfaz C	Clases C++
Generación	`flex fichero.l`	`flex -+ fichero.l`
Salida habitual	`lex.yy.c`	`lex.yy.cc`
Texto	`yytext`	`YYText()`
Longitud	`yyleng`	`YYLeng()`
Entrada	`yyin` de tipo `FILE*`	Flujo `std::istream`
Análisis	`yylex()`	`objeto.yylex()`

> `FlexLexer.h` declara `FlexLexer` y su clase derivada `yyFlexLexer`. El modo C++ lo incluye en el código generado.

Consulta: acciones avanzadas

Elemento	Efecto
`ECHO`	Copia el lexema a la salida
`yymore()`	Añade el próximo lexema al actual
`yyless(n)`	Conserva los primeros n bytes y devuelve el resto
`REJECT`	Prueba la siguiente mejor regla candidata
`yywrap()`	Al EOF: 0 para continuar, distinto de 0 para terminar

> FLEX no llama automáticamente a `yyerror()` si ninguna regla coincide. Para errores léxicos, escribe una regla explícita.

Materiales y referencias

Para practicar

`contador.l`, `tokens-inicio.l`, `tokens-solucion.l` y entradas de prueba en la carpeta `materiales`.

Documentación

Manual de FLEX · Patrones · Selección de reglas · Interfaz C++

Material de partida: «Análisis léxico con LEX», M. Jesús Rodríguez, Universidad de Granada (Seminario_Lex). Adaptación para Modelos de Computación.